

IA2

Internal Assessment 2 — Full Stack Web Development

Backend (Node.js / Express / MongoDB) · Frontend (React + TypeScript) · System Design

Duration:	3 Hours	Max Marks:	50
Name:		Roll No.:	

General Instructions

- Attempt all three questions. Each question belongs to a different layer of the same application — read all three before starting, as your backend design in Q1 should stay consistent with the frontend in Q2 and the system in Q3.
- Write clean, readable code. Indentation, naming, and comments carry marks.
- State any assumptions you make clearly at the top of your answer.
- Use of official documentation (Node.js, Express, Mongoose, React, TypeScript) is allowed. No AI code-generation tools or internet search for complete solutions.
- Submit backend code, frontend code, and the system design write-up as separate clearly labeled sections/files.

Marks Distribution

Q.No	Topic	Marks
1	Backend Development — Node.js, Express.js & MongoDB	20
2	Frontend Development — React with TypeScript	20
3	System Design	10
	Total	50

Section A — Backend Development (Node.js, Express.js, MongoDB) [20 Marks]

You are building the backend for “ShelfLife”, a library management platform used by a college library to manage books, members, and borrowing.

Q1. (20 Marks) Design and implement a REST API for the Library Management System described below.

Entities:

- Book — title, author, ISBN, genre, totalCopies, availableCopies
- Member — name, email, membershipId, joinedDate
- BorrowRecord — book (ref), member (ref), issueDate, dueDate, returnDate, status (issued / returned / overdue)

Tasks:

- a) Design Mongoose schemas for all three entities above, including appropriate data types, validation rules (e.g. required fields, unique ISBN/email), and references between collections.
- b) Implement the following REST endpoints using Express:
 - POST /api/books — add a new book
 - GET /api/books — list books, with support for pagination and filtering by genre
 - POST /api/members — register a new member
 - POST /api/borrow — issue a book to a member (must check availableCopies > 0, decrement it, and create a BorrowRecord)
 - POST /api/return/:borrowId — return a book (set returnDate, increment availableCopies, update status)
 - GET /api/members/:id/history — list a member's full borrow history
- c) Add middleware for: centralized error handling, request logging, and input validation (using a library such as Joi/Zod or manual checks).
- d) Implement a basic authentication layer: a /api/auth/login route issuing a JWT, and a middleware that protects the POST/issue/return routes so only logged-in librarians can use them.
- e) Explain (in 4–6 lines, as comments or a short note) how you would prevent two librarians from issuing the last copy of the same book at the same time (a race condition).

Deliverables: Express app with folder structure (routes/controllers/models/middleware), a README with setup + API list, and sample requests (curl or Postman collection) for each endpoint.

Section B — Frontend Development (React with TypeScript) [20 Marks]

You will build the frontend for the same “ShelfLife” system, consuming the API from Section A (you may mock the API with static/sample data if Section A isn't finished).

Q2. (20 Marks) Build a typed React + TypeScript frontend with the following screens and behavior.

Tasks:

- a) Define TypeScript interfaces/types for Book, Member, and BorrowRecord matching the backend schema, and a typed API client (using fetch or axios) with proper request/response typing.
- b) Build a Book List page: fetch and display books in a table/grid, with a search box (filter by title) and a dropdown filter by genre. Use useState/useEffect (or React Query) correctly, and show loading and error states.
- c) Build an “Issue Book” form: select a member and a book, submit to POST /api/borrow, show a success/error toast, and disable the submit button while the request is in flight.
- d) Build a Member History page showing a member's BorrowRecord list, with a visually distinct badge for overdue items (dueDate < today and not yet returned).
- e) Create at least one reusable generic component, for example a typed <DataTable<T>> or <Select<T>> component that works for both books and members.
- f) Add client-side route protection: redirect to a /login page if no auth token is present, using React Router.

Deliverables: React + TypeScript project (Vite or CRA), component folder structure, and a short note on your state-management choice (local state vs Context vs a library) and why.

Section C — System Design [10 Marks]

Q3. (10 Marks) Design the system for scale.

ShelfLife has been adopted by a university network and must now support 500 campus libraries, ~2 million members, and heavy usage during the first week of every semester (10× normal traffic for book search and issue/return operations).

Answer the following:

- a) Draw (describe in words/diagram) a high-level architecture: client, API layer, database, cache, and any other components you'd add (load balancer, queue, CDN, etc.).
- b) Would you keep a single MongoDB cluster or shard it? If sharding, propose a shard key for the Book and BorrowRecord collections and justify it.
- c) Identify the single most read-heavy operation in this system and describe a caching strategy for it (what you'd cache, cache invalidation trigger, and TTL).
- d) The “issue book” operation must never let availableCopies go negative even under concurrent requests at scale. Describe one mechanism to guarantee this (e.g. atomic DB operations, optimistic locking, distributed locks, or a queue) and explain why you chose it over the alternatives.
- e) How would you handle the 10× traffic spike during semester week without over-provisioning infrastructure year-round?

Deliverables: A one-to-two page written answer with a labeled architecture diagram (hand-drawn/scanned, or any diagramming tool) and justifications for each design choice.